MLOps

MLOps Assignment 1 Report

Author: Rouzif Rasheed Moopan - 2025AE05986

Introduction

This report consolidates the work done towards the ML Ops Assignment 1 on the topic End-to-End ML Model Development, CI/CD, and Production Deployment.

Deliverables

Code base:

- Github Repository:

  https://github.com/rouzif/WILP_MLOps_Assignment_1.git

- Google drive link to zip file: https://drive.google.com/file/d/1cPkvIikISGDPfxwTEM7p-eRJ-UbYEoZq/view?usp=drive_link

Video Demo:

- Google drive link (public):

  https://drive.google.com/drive/folders/1NNRhQUxDLTwRFHOjI8tzsuU6KKKgWbsA?usp=sharing

Screenshots:

- PDF with explanations:

  https://drive.google.com/file/d/1EyDcWI1Sv93yIrcDQchsEl16Qc_d0zg3/view?usp=drive_link

- Zip file:

https://drive.google.com/file/d/1Rmhpw1bFcTvjz6xWedShBLiVL6aVdAcl/view?usp=drive_link

- Github:

  https://github.com/rouzif/WILP_MLOps_Assignment_1/tree/main/screenshots

Instructions on setting up and accessing API locally (Also provided at the end of this doc):

- Google drive link:

  https://drive.google.com/file/d/1G0E-Gb0yI5B-klJZDIuHzImHLykJP0gq/view?usp=drive_link

Running on Prayogshala lab:

- Workspace Location: /home/s2025ae05986/workspace/2025AE05986/MLOps_Assignment_1/WILP_MLOps_Assignment_1/
- Prayogshala Lab environment usage:

  https://drive.google.com/file/d/1QJDDUZmi_iSLF13kU26LoiDAavgEkvxT/view?usp=drive_link

Task Deepdives

Data Acquisition & Exploratory Data Analysis (EDA)

Dataset details:

- Heart Disease UCI Dataset
- 303 patient records across 14 features
- Features cover Age, sex, clinical attributes etc.
- Target variable “Heart disease flag” is set to be 1 if the patient falls in any of the heart disease categories (heart_disease != 0)

Loading data:

- Script: `src/data/load_data.py`
- Methodology: Fetch directly from repo using ucimlrepo library
- Fingerprinting: `src/utils/data_fingerprint.py`
- Deterministic Fingerprinting using SHA-256 cryptographic checksum hashing

Missing value imputation:

- Script: `src/data/missing_values.py`
- Median Imputation for continuous attributes (chol, trestbps, thalach, oldpeak)
- Mode Imputation for Categorical Attributes (ca, thal)

Encoding:

- Script: `src/data/encode.py`
- One hot encoding for categorical features (number of unique values <=5)
- N-1 categories retained in encoding for a feature with N categories

Exploratory data analysis:

- Script: `src/data/visualize.py`
- Frequency bar charts for categorical variables
- Histograms for continuous variables
- Boxplots for continuous variables vs target
- % Heart disease rate bar chart for categorical variables
- Correlation heatmap

Data Partitioning:

- Data split into test (30%) and train (70%)
- Splitting done before any preprocessing steps to prevent leakage of data in terms of preprocessing parameters

Feature Engineering & Model Development

Custom Preprocessor Architecture (`src/data/preprocessor.py`):

- Implemented an sklearn-compatible custom transformer DataPreprocessor inheriting from BaseEstimator and TransformerMixin to guarantee strict isolation between train and test distributions, preventing data leakage.
- Categorical Feature Encoding: Categorical variables (cp, restecg, slope, thal) encoded using one-hot encoding.
- Standard Feature Scaling: Continuous variables (age, trestbps, chol, thalach, oldpeak) normalized via StandardScaler to zero mean and unit variance.

Model Training & Benchmarking (`src/model/train_model.py`):

- Implemented classification algorithms under 5-Fold Cross-Validation - Random Forest Classifier and Gradient Boosting Classifier
- Parameters searched: n_estimators, max_depth, min_samples_leaf for Random Forest, and n_estimators, max_depth, learning_rate for Gradient Boosting.
- Model selection done based on Accuracy, Precision, Recall, and Aread Under ROC Curve.
- Random Forest model gives the best test AUC when cross-validated with param grid search "n_estimators": [200, 400], "max_depth": [3, 5], "min_samples_split": [5, 10]
- Model evaluation metrics:
    - Accuracy: 85.7%
    - Precision: 83.7%
    - Recall: 85.7% (prioritized to minimize clinical false negatives)
    - ROC-AUC: 92.7%

Experiment Tracking

MLflow Experiment Lifecycle:

- Integrated full experiment tracking via MLflow.

- Parameters Logged: Hyperparameters including n_estimators, max_depth, learning_rate, C, and subsample ratio.
- Metrics Logged: Accuracy, Precision, Recall, F1-Score, and ROC-AUC across training, 5-fold cross-validation, and hold-out test splits.
- Artifacts Logged: Model binary (model.joblib), preprocessor transformer (preprocessor.joblib), MLflow input/output model signatures, evaluation JSONs (eval_train.json, eval_test.json), ROC Curves, and Precision-Recall plots.

Model Registry & Alias Promotion: Automatically registered candidate models in the MLflow Model Registry and assigned the @champion alias to the top-performing model based on test set ROC-AUC.

Model Packaging & Reproducibility

Model Serialization (`model/champion/`): Exported champion artifacts into two formats:

- Standard .joblib binaries for native Python execution: `model/champion/model.joblib, model/champion/preprocessor.joblib`
- Skops format for pickle-free secure model loading: `model/champion/model/, model/champion/preprocessor`

Environment & Dependency Management: Pinned dependency versions in `requirements.txt` to ensure reproducible execution across local workstations, Docker containers, and Kubernetes pods.

CI/CD Pipeline & Automated Testing

Pytest Automated testing (`tests/`):

- `test_data_processing.py`: Validates missing value imputation logic, numeric constraints, and scaling statistics.
- `test_preprocessor.py`: Tests the preprocessor module
- `test_train_model.py`: Tests model evaluation metrics generation and binary serialization workflows.
- `test_api.py`: Tests FastAPI endpoints, validating HTTP 200 responses for valid payloads and HTTP 422 for invalid schemas.

GitHub Actions Workflow (`.github/workflows/ci.yml`):

- Triggered on every git push or pull_request to the main branch
- Stage 1 (Linting): Code quality enforcement using flake8.
- Stage 2 (Unit Testing & Coverage): Executes pytest tests/ -v --cov=src with automated coverage reporting.
- Stage 3 (E2E Pipeline Check): Verifies full pipeline execution via python main.py --pipeline train.
- Stage 4 (Uploading artifacts): Uploads model and preprocessor joblib files
- Stage 5 (Upload tracking artifacts): ML flow database, Performance metrics on test and train data, model training plots

Model Containerization

FastAPI Model Serving API (`app/main.py`):

- `GET /health`: Liveness health probe returning API operational status.
- `POST /predict`: Inference endpoint accepting patient JSON payloads, performing inline preprocessor transformation, and returning class prediction along with prediction probability confidence.
- `GET /metrics`: Exposed Prometheus metric instrumentation via prometheus-fastapi-instrumentator.

Docker Image Architecture (`Dockerfile`):

- Built using official `python:3.13.7-slim` base image to minimize container footprint and reduce attack surface.
- Implemented multi-layer build caching by isolating dependency installation (`pip install -r requirements.txt`) prior to copying application source code.

Local Validation: Verified container runtime locally via docker build and docker run mapping port 8000.

Production Deployment

Kubernetes Cluster Orchestration (`k8s/`):

- Namespace (`k8s/namespace.yaml`): Provisioned isolated heart-disease-mlops namespace.
- Deployment (`k8s/deployment.yaml`): Configured high-availability deployment running 2 Pod replicas. Included CPU resource requests (250m) and limits (500m), RAM requests (256Mi) and limits (512Mi), alongside livenessProbe and readinessProbe checking GET /health.
- Service (`k8s/service.yaml`): Exposed application via LoadBalancer mapping external port 8000 to container port 8000.

Deployment Verification: Verified pod health and LoadBalancer routing via kubectl get pods,svc -n heart-disease-mlops and external curl verification.

Monitoring & Logging

Structured HTTP Request Logging (`app/main.py`): Implemented custom `@app.middleware("http")` capturing client IP, HTTP method, endpoint path, status code, and latency in milliseconds to stdout.

Prometheus Telemetry Scraper (`monitoring/prometheus.yml`): Configured Prometheus scraping target to collect metrics from /metrics every 5 seconds.

Grafana Operational Dashboard (`monitoring/docker-compose.monitoring.yml`): Deployed integrated Grafana monitoring stack with visual panels tracking:

- API Request Rate (RPS):

  ```
  sum(rate(http_requests_total[1m])) by (handler, status)
  ```

- p95 Latency:

  ```
  histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket[1m])) by (le))
  ```

- HTTP Status Breakdown & Error Rate:
  `sum(rate(http_requests_total{status=~"4..|5.."}[1m]))`

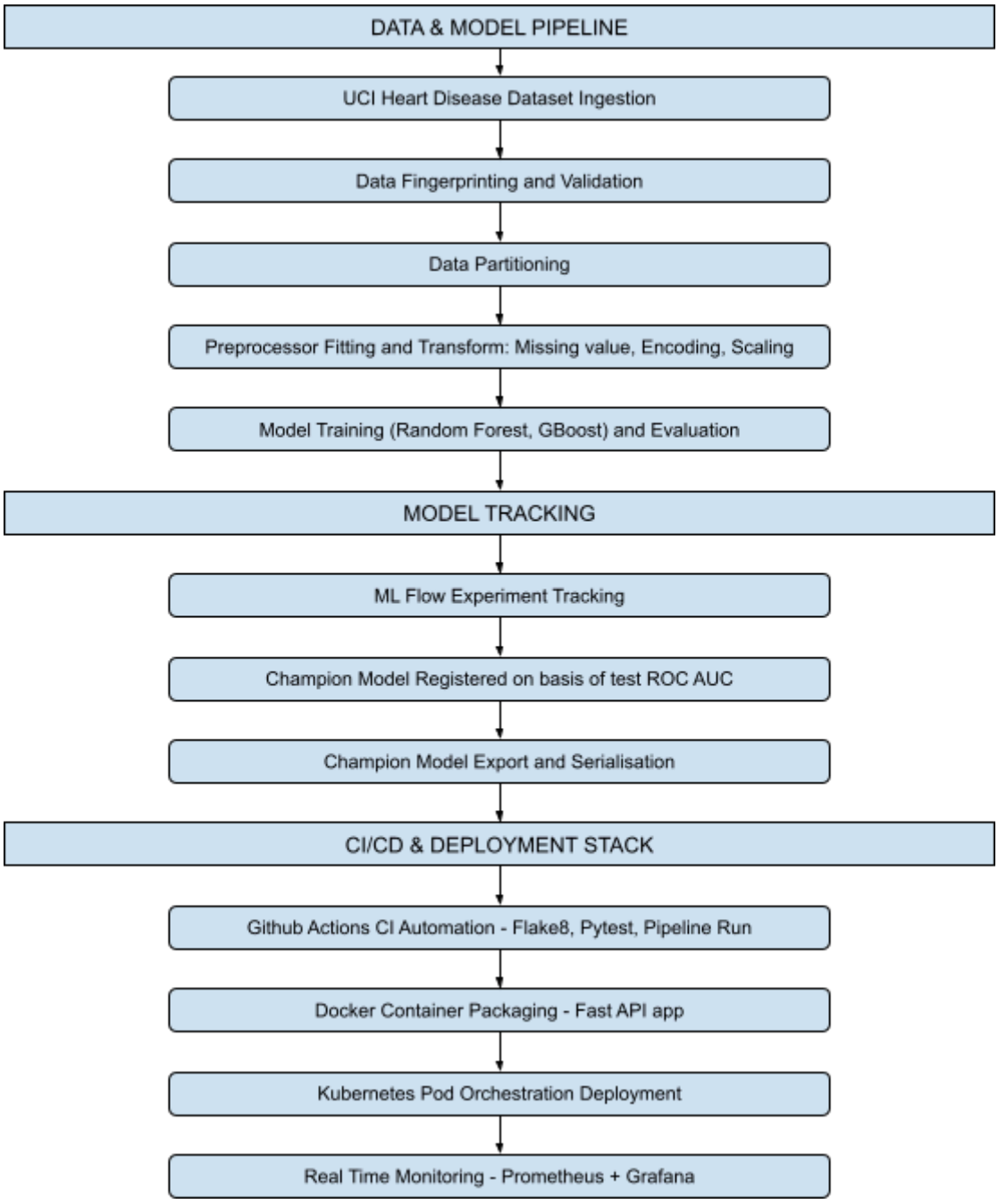
Traffic Load Generation (`monitoring/generate_traffic.py`): Automated load testing script triggering real-time time-series metric graphs.

System Architecture & Repository Structure

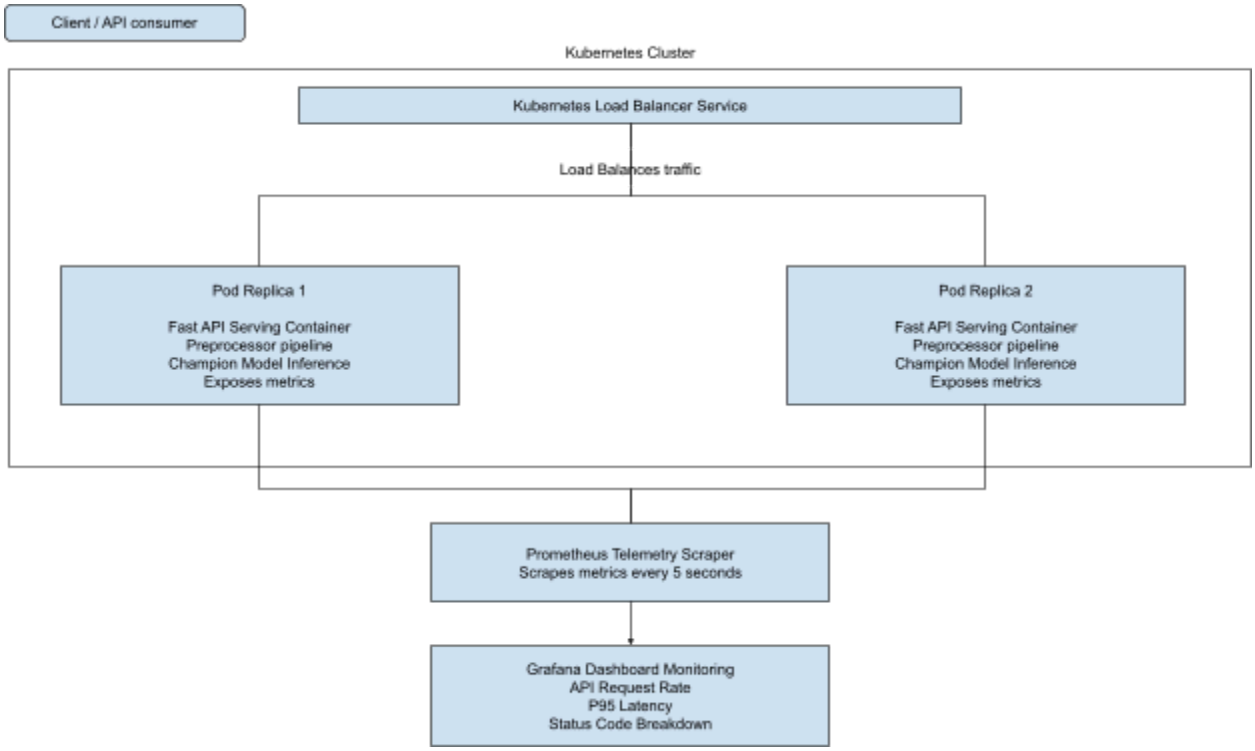
Project Repository set up

```
WILP_MLOps_Assignment_1/
├── .github/workflows/
│   └── ci.yml
├── app/
│   ├── main.py
│   └── schemas.py
├── config/
│   └── config.yaml
├── k8s/
│   ├── deployment.yaml
│   ├── namespace.yaml
│   └── service.yaml
├── model/
│   └── champion/
├── monitoring/
│   ├── docker-compose.monitoring.yml
│   ├── generate_traffic.py
│   └── prometheus.yml
├── src/
│   ├── data/
│   ├── model/
│   ├── utils/
│   └── workflow/
├── tests/
├── Dockerfile
├── main.py
└── requirements.txt
```

End-to-End System Pipeline



Production Deployment Topology



Instructions to set up and run

Step 1: Prerequisites & Environment Setup

Ensure the target machine has the following dependencies installed prior to execution:

- **Git** (v2.30+)
- **Python** (v3.10+)
- **Docker Desktop** (with Kubernetes enabled) or **Minikube**
- **kubectl CLI**
    - Download the latest stable release binary
      `curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"`
    - Make the binary executable
      `chmod +x kubectl`
    - Move the binary to /usr/local/bin
      `sudo mv kubectl /usr/local/bin/`
    - Check installation
      `kubectl version --client`

Step 1.1: Clone the Repository

Retrieve the project source code and navigate into the working directory:

```
git clone https://github.com/rouzif/WILP_MLOps_Assignment_1.git
cd WILP_MLOps_Assignment_1
```

Step 1.2: Create and Activate Virtual Environment

Initialize an isolated Python virtual environment:

- **macOS / Linux:**
  ```
  python3 -m venv .venv
  source .venv/bin/activate
  ```

- **Windows (PowerShell):**
  ```
  python -m venv .venv
  .venv\Scripts\Activate.ps1
  ```

Step 1.3: Install Dependencies

Upgrade the core package manager and install all required runtime and development packages:

```
pip install --upgrade pip
pip install -r requirements.txt
```

Step 2: Local Testing & ML Pipeline Execution

Step 2.1: Run Unit Tests

Execute the automated test suite to ensure system integrity:

```
python -m pytest tests/ -v
```

Step 2.2: Execute Training Pipeline

Run the end-to-end model training workflow:

```
python main.py --pipeline train
```

Step 2.3: (Optional) Launch MLflow Tracking Server

To inspect experiment logs, run parameters, and performance metrics visually:

```
mlflow server --backend-store-uri sqlite:///data/mlflow.db
--default-artifact-root ./mlruns --host 127.0.0.1 --port 5001
```

Access URL: Navigate to `http://localhost:5001` in your web browser.

---

3. Docker Containerization

Step 3.0: Navigating to working directory in new terminal

Open a new terminal and navigate to working directory

```
cd WILP_MLOps_Assignment_1
```

Step 3.1: Build Container Image

Package the application into a standard container image:

```
docker build -t heart-disease-api:v1 .
```

Step 3.2: Run Container Locally

Instantiate the container service in detached mode:

```
docker run -d --rm --name heart-disease-service -p 8000:8000
heart-disease-api:v1
```

Step 3.3: Verify Endpoints

Validate container operational readiness using the following interfaces:

- **Health Probe:**
  `curl -X GET http://localhost:8000/health`
- **Inference Endpoint:**
  ```
  curl -X POST http://localhost:8000/predict -H "Content-Type:
  application/json"  -d '{"age": 52, "sex": 1, "cp": 0, "trestbps": 125,
  "chol": 212, "fbs": 0, "restecg": 1, "thalach": 168, "exang": 0,
  "oldpeak": 1.0, "slope": 2, "ca": 2, "thal": 3}'
  ```

- **Swagger API Documentation:**
  Access interactive API docs at `http://localhost:8000/docs`

Step 3.4: Stop Local Container

Terminate the container instance prior to Kubernetes deployment:

```
docker stop heart-disease-service
```

---

4. Kubernetes Deployment

Step 4.1: Enable Cluster

Ensure your local Kubernetes engine (Docker Desktop Kubernetes or Minikube) is actively running:

```
minikube start
kubectl cluster-info
```

Step 4.2: Build docker image directly inside Minikube's docker daemon

```
eval $(minikube docker-env)
docker build -t heart-disease-api:v1 .
```

Step 4.3: Apply Kubernetes Manifests

Deploy the namespace, application pods, and routing service:

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
```

Step 4.4: Verify Cluster Resources

Confirm that all resources have been instantiated and pods are in a `Running` state:

```
kubectl get pods,svc,deploy -n heart-disease-mlops
```

Step 4.5: Port-Forward API Service

Expose the Kubernetes service to your local workstation:

```
kubectl port-forward svc/heart-disease-api-service 8000:8000 -n
heart-disease-mlops
```

---

5. API Access and Endpoint testing

Health Probe:

```
curl -X GET http://localhost:8000/health
```

Inference Endpoint:

```
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json"
-d '{"age": 52, "sex": 1, "cp": 0, "trestbps": 125, "chol": 212, "fbs": 0,
"restecg": 1, "thalach": 168, "exang": 0, "oldpeak": 1.0, "slope": 2, "ca": 2,
"thal": 3}'
```

Swagger API Documentation:

Access interactive API docs at `http://localhost:8000/docs`.